Homework 5

Statistical Modeling and Learning

due: **Sunday, November 8, 2026, 5:00p**

Instructions: Please submit solutions on Populi as a professionally compiled PDF (for example, from a knitted R Markdown file or Jupyter notebook). For handwritten work, please scan your solution pages and include them in the final PDF. Homeworks can be completed individually or in groups of up to three people. Names of all group members must be included on the write-ups.

Problem 1: Network Structure. No software needed. Use the notation from class: in layer ℓ, $z_j^{(\ell)} = \sum_k W_{jk}^{(\ell)} a_k^{(\ell-1)} + b_j^{(\ell)}$ and $a_j^{(\ell)} = \phi(z_j^{(\ell)})$, with $a^{(0)} = x$.

a. Draw a neural network with 2 inputs (x_1, x_2), one hidden layer with 2 hidden units, and 1 output $\hat{y}$. Label every weight and bias on your drawing (e.g., $W_{11}^{(1)}$, $W_{12}^{(1)}$, $b_1^{(1)}$, …, $W_{11}^{(2)}$, $b_1^{(2)}$), and show how the output is a composition of functions from the inputs, to the hidden layer, to the output.

b. For the network in part (a), with activation ϕ in the hidden layer and no activation on the output (regression), write $\hat{y}$ explicitly as a function of x_1 and x_2. Then rewrite it in the form $\hat{y} = \beta_0 + \sum_{k=1}^{2} \beta_k \, \phi(b_k + w_k^\top x)$ from class, saying which weights play the roles of β_0, β_k, b_k, and w_k.

c. *Forward propagation by hand.* Suppose the hidden layer uses ReLU, $\phi(z) = \max(0, z)$, and the weights are

$$W^{(1)} = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 2 & -1 \end{pmatrix}, \quad b^{(2)} = 0.5.$$

For the input $x = (x_1, x_2) = (1, 2)$, compute $z^{(1)}$, $a^{(1)}$, and $\hat{y}$. Show your work. Which hidden unit is "switched off" for this input?

d. *Why activation functions matter.* Now drop the activation, so that $\phi(z) = z$. Show algebraically that the network collapses to a single linear regression $\hat{y} = c_0 + c_1 x_1 + c_2 x_2$. Using the weights from part (c), compute c_0, c_1, and c_2, and the prediction at $x = (1, 2)$. In one or two sentences, explain why a network with many hidden units (and a nonlinear activation) can approximate complex nonlinear relationships, whereas a linear model cannot.

e. *Counting parameters.* A network with p inputs, one hidden layer of K units, and one output has $(p + 1)K + (K + 1)$ parameters. Explain where each term comes from. How many parameters does a network with $p = 11$ inputs and $K = 5$ hidden units have? (You will check this number in Problem 3.) How many parameters does the network pictured on the "Basic concepts: neurons" slide have (3 inputs, two hidden layers of 4 units each, 1 output)?

f. How many units should the output layer have, and what activation should it use, for (i) predicting a house price, (ii) predicting whether an email is spam, and (iii) classifying a satellite image into one of 4 land-use classes? For (ii), explain why a network with no hidden layer and a single sigmoid output unit is exactly the logistic regression model we studied earlier in the course.

Problem 2: Gradient Descent and the Chain Rule. Consider a single neuron with one input: $\hat{y} = \sigma(wx + b)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid. For one data point (x, y), the squared-error loss is $L = \frac{1}{2}(\hat{y} - y)^2$.

a. Let $z = wx + b$. Write L as a composition of three functions ($w, b \mapsto z \mapsto \hat{y} \mapsto L$). Using the chain rule, derive $\dfrac{\partial L}{\partial w}$ and $\dfrac{\partial L}{\partial b}$ in terms of $\hat{y}$, y, x, and $\sigma'(z)$. (Recall: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.)

b. For $(x, y) = (2, 1)$ and $(w, b) = (0.5, 0)$, compute z, $\hat{y}$, L, and the gradients $\dfrac{\partial L}{\partial w}$ and $\dfrac{\partial L}{\partial b}$.

c. Perform one gradient descent step with learning rate $\alpha = 0.5$. Report the new (w, b) and the new $\hat{y}$. Did $\hat{y}$ move toward $y = 1$?

d. In `R` or `Python`, implement gradient descent for this neuron. Use the data point $(x, y) = (2, 1)$, initial values $(w, b) = (0.5, 0)$, and $\alpha = 0.5$, and run 20 iterations. Report the final (w, b) and plot L against the iteration number.

e. Go back to your network from Problem 1(a), with a sigmoid hidden layer and the same squared-error loss. In one or two sentences, describe where the chain rule is used when computing $\partial L / \partial W_{11}^{(1)}$, a weight in the *first* layer. Why does this computation run "backwards" from the output (backpropagation)?

Problem 3: Neural Network Regression. Download the file `wine.csv` from the course website. This data set contains chemical properties of red and white wines. The outcome is `quality` (an integer score from 0–10), and the predictors are the 11 chemical variables `fixed.acidity` through `alcohol`. Use the `nnet` package.

a. Split the data 70/30 into training and test sets, running `set.seed(42)` before you sample the training rows. Scale all predictors to mean 0 and standard deviation 1, using the *training* set means and standard deviations for both sets. In one sentence, why should the test set be scaled with the training set's means and standard deviations rather than its own?

b. Fit a neural network with one hidden layer of 5 units: `nnet(..., size = 5, linout = TRUE, maxit = 500, trace = FALSE)`, running `set.seed(42)` first. How many weights does the fitted model have (`length(fit$wts)`)? Does it match your answer to Problem 1(e)? What does `linout = TRUE` do, in terms of the output layer's activation?

c. Compute the test-set RMSE of your network and of a linear regression of `quality` on the same predictors. Which predicts better? (Note: neural networks are sensitive to random initialization, so your results may vary slightly with the seed.)

d. Refit the network with `size` = 1, 2, 5, 10, and 20 hidden units (run `set.seed(42)` before each fit). Make a table of the number of parameters, training RMSE, and test RMSE for each. What happens to training RMSE as the network grows? To test RMSE? Connect what you see to the bias–variance tradeoff.

e. Weight decay adds a penalty $\lambda \sum w^2$ to the loss, just like ridge regression. Refit the 20-unit network with `decay = 0.1` and `decay = 1`. Add these fits to your table. Does weight decay fix the problem you saw in part (d)? In one or two sentences, explain why.

f. In one or two sentences, discuss when you might prefer a neural network over a linear model, and when you might not.

Problem 4: Convolutional Neural Networks.

By hand. A single-channel 6×6 image X and a 3×3 filter F are

$$X = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}, \qquad F = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}.$$

a. Slide the filter across the image (stride 1, no padding). At each position, multiply the filter entrywise by the 3×3 patch underneath it and add up the nine products. What are the dimensions of the resulting feature map? Compute all of its entries.

b. Apply ReLU and then 2×2 max pooling (stride 2) to your feature map. What is the result?

c. What kind of pattern in the image does this filter respond to? Where are the largest values in your feature map, and which part of the image do they correspond to? In one or two sentences, explain why a CNN *learns* the entries of its filters rather than having us choose them.

d. *Counting parameters.* Color images have 3 channels (red, green, blue). Consider the CNN below for 64×64 color images and 4 classes:

Layer	Description
Conv 1	16 filters of size $3 \times 3 \times 3$, padding 1, then ReLU
Pool 1	2×2 max pooling
Conv 2	32 filters of size $3 \times 3 \times 16$, padding 1, then ReLU
Pool 2	2×2 max pooling
Fully connected	flatten, then 64 hidden units with ReLU
Output	4 units with softmax

Give the dimensions (channels $\times$ height $\times$ width) of the output of each of the first four layers. (Padding 1 keeps the height and width the same through a 3×3 convolution.) Then count the parameters in each layer, where each filter has one bias. Compare the parameter count of Conv 1 with that of a fully connected layer mapping the flattened $3 \times 64 \times 64$ image directly to 64 hidden units. What two ideas explain the difference?

Coding: image classification. Train a neural network to classify satellite images from the EuroSAT land-use data set, using PyTorch in a Jupyter notebook. Starter code is in the notebook `HW5_cnn_eurosat.ipynb` in the `code` folder of the course website. It downloads the data through `torchvision`, keeps 250 images from each of 4 classes (`Forest`, `Highway`, `Residential`, `SeaLake`), makes an 80/20 train–test split, and defines a training loop and the CNN from part (d).

Getting set up with PyTorch. The simplest option is Google Colab, which needs no installation. Sign in with a Google account and click **this link to open the notebook in Colab**. (You can also go to `https://colab.research.google.com`, choose *File* $\rightarrow$ *Upload notebook*, and upload `HW5_cnn_eurosat.ipynb` from the course website.) PyTorch is already installed there, and the

notebook downloads the images itself. Run the cells from top to bottom with Shift+Enter; training runs on Colab's standard CPU in about a minute. If you would rather work on your own computer, install PyTorch and Jupyter from a terminal with `pip install torch torchvision matplotlib jupyter`, then run `jupyter notebook` and open the notebook.

e. Train the CNN and report its test-set accuracy. Include a grid of example test images with their predicted and actual classes, and a confusion matrix cross-tabulating actual against predicted class on the test set.

f. Now train a feedforward network on the flattened images (no convolutions): flatten, one hidden layer of 64 units with ReLU, and 4 outputs. Use the same training loop. Report its number of parameters and its test accuracy, and compare both with the CNN. Which network has more parameters? Which classifies better? Use your answer to part (d) to explain why.

g. Which pair of classes does the CNN confuse most often? Look at some of the misclassified images and give a plausible reason.

Problem 5: Difference-in-Differences as a Variable Interaction. This problem estimates the effect on homicide of castle-doctrine ("stand your ground") laws, which expand the legal justifications for using lethal force in self-defense. Install the `causaldata` package and load the data with `data(castle, package = "causaldata")`. The data come from Cheng and Hoekstra (2013) and cover the 50 states from 2000 to 2010. The variable `post` equals 1 in the years a state's castle-doctrine law is in effect. Use `l_homicide` (the log homicide rate) as the outcome Y_{it}.

Create a variable `treated` equal to 1 if a state ever adopted a castle-doctrine law during the sample period (that is, `post` equals 1 for at least one year in that state), and 0 otherwise. Control states never adopt, so give them a comparable "late" period: `post_did` equals 1 if (`treated` = 1 and `post` = 1) or (`treated` = 0 and `year` $\geq$ 2007), and 0 otherwise.

a. Compute the four group means of Y_{it}: treated states before and after adoption, and control states in the early and late periods. Compute the before–after difference for treated states and for control states. What is the difference-in-differences (DiD) estimate, and what does it represent?

b. Fit the regression $Y_{it} = \beta_0 + \beta_1 \,\text{Treated}_i + \beta_2 \,\text{Post}_{it} + \tau\,(\text{Treated}_i \times \text{Post}_{it}) + \varepsilon_{it}$, using `post_did` for Post. Show that $\hat{\tau}$, the coefficient on the interaction, equals your DiD from part (a). Interpret each of β_0, β_1, β_2, and τ in terms of the four group means.

c. States adopted their laws in different years, so a single "late" period for control states is a rough approximation. A better regression replaces Treated and Post with a dummy for every state and every year ("two-way fixed effects"): `lm(l_homicide ~ post + factor(sid) + factor(year), data = castle)`. Report the coefficient on `post`. Because the outcome is in logs, interpret it as an approximate percent change in the homicide rate. How does it compare with part (b)?

d. State the parallel-trends assumption that DiD relies on, in your own words. Make a plot of the average of Y_{it} by year, separately for treated and control states, and use it to discuss whether the assumption is plausible here.